

Elliott Wave Pattern Agent

Review swing points and candidate wave patterns, then understand confidence and invalidation.

Release 2.1.1-source.20260928 | Updated October 3, 2026

Scope and limitations

This is downloadable source software. Hosting, trading funds and external services are separate. Build and signal checks are not certification for unattended real-money execution. Complete venue and recovery validation for your installation. Some packaged agent INSTALL/manifest files retain the older word “candidate”. This guide is tied to the registered September 28 archive by checksum; publication does not certify venue performance.

Fibonacci tolerances are percentages; retracement values such as 0.618 are ratios. Degree labels describe the module output and do not fetch multiple candle series. Chart overlays and guaranteed wave forecasts are not included.

Requirements

A working Core/Trader dashboard in the manifest range $\geq 2.1.0$ $< 2.2.0$. The browser-only Core 2.0 template is incompatible.

The module manifest declares Node ≥ 20 ; the complete shipped dashboard requires Node 22.3 or newer.

Completed, time-ordered OHLCV candles with millisecond timestamps, finite positive prices and valid volume, supplied by your runtime.

A backup and an isolated paper/research environment. The strategy computes signals; the dashboard supplies market data, order handling and risk controls.

Purchase and download

1. Sign in to the account used at checkout. Open Account and locate the confirmed order. If payment is processing, wait for fulfillment rather than buying again.
2. Generate a fresh product Download link. Save the ZIP, receipt, release version and checksum. Expired links can be regenerated from the account.
3. Compare the archive SHA-256 with the release recorded below. Extract into a new directory; retain the original archive for rollback.
4. Open README.md, RUNBOOK.md or INSTALL.md, manifest.json where included, and LICENSE.md before changing an installation.

Archive SHA-256: 32f85ae0a257a486d3c29df982dd905a7a64f8032969852c549e3f9624ff7f87

Installation

1. Check whether you already own the strategy: all six are included in Trader; Darvas is also included in Core.
2. Back up the database, settings and plugins. Pause new entries and reconcile positions before changing the installation.
3. Extract the module so manifest.json sits directly at plugins/elliott-wave-agent/manifest.json. Check plugins/elliott-wave-agent/dist/strategy.js exists; do not nest the product directory twice.
4. Restart the dashboard and inspect loader errors. Select strategy ID elliott-wave, a supported interval and the intended market. Start paused with no funds allocated.
5. Use the manifest defaults for the first comparison. Inspect decisions and then independently validate the runtime trade lifecycle before any unattended execution.

Run from the dashboard root

```
node -e "const p=require('./plugins/elliott-wave-agent/dist/strategy.js'); if(!p.strategy) throw Error('strategy export missing'); console.log('strategy export present')"
```

How the strategy works

The module finds local swing highs/lows and checks programmed impulse-wave and Fibonacci relationships. Candidate wave counts can change as more candles arrive. It is one rule-based interpretation, not a definitive count of every Elliott pattern.

Strategy ID: elliott-wave | Path: plugins/elliott-wave-agent

Supported intervals: 1h, 4h, 1d

Exact shipped defaults

Defaults are comparison inputs, not recommended trading parameters. Consult the source validation rules before changing them.

Setting	Default	Meaning
swingLookback	5	Periods for pivot detection
minWaveSize	1.5	Minimum wave size as %
maxWaveDeviation	15	Max deviation from ideal Fibonacci ratio %
wave3MinMultiplier	1	Reserved legacy field; not read by the shipped signal calculation. Changing it does not alter decisions in this release.
wave3IdealMultiplier	1.618	Wave 3 ideal Fibonacci ratio
wave5IdealRatio	1	Wave 5 vs wave 1 ratio

Setting	Default	Meaning
fibRetrace618	0.618	Reserved legacy field; not read by the shipped signal calculation. Changing it does not alter decisions in this release.
fibRetrace50	0.5	Reserved legacy field; not read by the shipped signal calculation. Changing it does not alter decisions in this release.
fibRetrace382	0.382	Reserved legacy field; not read by the shipped signal calculation. Changing it does not alter decisions in this release.
minConfidence	60	Minimum confidence to signal
degreeLabels	["Minute", "Minor", "Intermediate"]	Reserved legacy field; not read by the shipped signal calculation. Changing it does not alter decisions in this release.

Step-by-step practice tutorial

1. Verify the installed module

Follow the install steps above, then run the export check from the dashboard root. Expected result: strategy export present. This checks loading only; it does not submit an order.

2. Prepare comparable inputs

Select 1h, 4h, 1d as supported by the manifest. Supply one completed candle series for one symbol, retaining timestamps, OHLCV, source and network. Multiple listed intervals do not mean automatic multi-timeframe fetching.

3. Run the strategy-specific exercise

Use completed 1h candles and shipped defaults. Record the candidate pattern and confidence; compare it with the configured minConfidence of 60. Inspect the pivot lag: swingLookback needs candles on both sides of a candidate pivot, so the latest candle is not automatically a confirmed swing.

4. Interpret the result

Read the returned reasoning and indicator object rather than treating every chart swing as a setup. Compare a neutral result with a pattern that satisfies the programmed rules. Check all proposed protection prices before any execution.

5. Validate the runtime separately

Confirm ownership, sizing, fees, entry fill, stop/target, modifications, exit and realized P&L on testnet. Include restart, partial fill, rejected cancellation and lost-response reconciliation. A signal test is not an execution test.

Acceptance checklist

[] Archive checksum and version match the guide.

[] Install completes without suppressing errors; the module loads from its declared path.

[] Correct owner, market, network and candle interval are recorded.

[] Decision timestamp, direction, reasoning and parameters are captured; neutral is accepted.

[] Settings and ownership persist after restart; backup restore is tested in isolation.

[] Before live execution: entry, fill, protection, modification, exit, fees and P&L reconcile; partial fills, cancellation failure and lost responses are tested.

Read the returned reasoning and indicator object rather than treating every chart swing as a setup. Compare a neutral result with a pattern that satisfies the programmed rules. Check all proposed protection prices before any execution.

Maintain and customize

Keep a copy of the immutable ZIP, lockfile, private configuration and a database backup before an update. Store encrypted backups separately and prove a restore in an isolated compatible instance.

Pause new entries and reconcile existing venue orders and positions before maintenance. Pausing entries is not the same as closing positions. Removing a plugin does not close exchange positions.

Monitor the last successful cycle, data age, rejected orders, protection status, reconciliation gaps, database backups and disk space. Missing telemetry is an unknown state.

For rollback, stop the new runtime, reconcile the venue, restore a compatible application/schema pair in isolation and verify ownership and protection before resuming. A database restore cannot reverse a trade or transfer.

Customize readable source in a separate branch. Preserve the original defaults, rebuild any changed compiled module, and test the same inputs before and after. Follow LICENSE.md; source ownership does not imply unrestricted redistribution.

Troubleshooting

Payment confirmed but download missing

Check order status in Account and contact support with the order reference if fulfillment is delayed. Do not make another purchase to repair access.

Download link expired

Generate a fresh link from the purchasing account. The documentation downloads are public; the paid software archive remains entitlement-protected.

Strategy absent from the selector

Confirm manifest.json is directly inside the manifest installPath, the entryPoint exists, compatibility matches and the loader has no errors. Restart after installation; do not add another nesting level.

No trade despite available capital

Inspect the latest decision, candle history and freshness, symbol, interval, network, existing positions and risk rejection. Neutral is a valid result.

Build or database initialization fails

Keep the exact failed command and redacted diagnostic. Check Node, psql, Supabase settings and owner verification. Do not bypass types, authentication or schema integrity checks.

Balances disagree or an order outcome is unknown

Compare the same account and network at the venue, including fills, fees, margin and open orders. Reconcile before retrying; a lost response does not mean the order failed.

Help and supporting video lessons

<https://www.civalsystems.com/docs/products/elliott-wave-agent>

Supporting videos show the broader demo interface. The product-specific written tutorial matches this archive.

Contact <https://www.civalsystems.com/contact> with your order reference, product/version, OS, Node version and redacted error. Never send secrets. Refund requests: <https://www.civalsystems.com/refund-request>.

Source references

`elliott-wave-agent/INSTALL.md`

`elliott-wave-agent/LICENSE.md`

`elliott-wave-agent/README.md`

`elliott-wave-agent/dist/strategy.js`

`elliott-wave-agent/manifest.json`

`elliott-wave-agent/src/strategy.ts`